

BÀI KIỂM TRA LAB

Xây dựng hệ thống Microservices với Spring Cloud

Student Service · Course Service · API Gateway

Spring Boot 4.x Nền tảng	2025.x Spring Cloud	MS SQL Server Database	90 phút Thời gian
------------------------------------	-------------------------------	----------------------------------	-----------------------------

1. Mục tiêu & Tổng quan

Sau khi hoàn thành bài lab, sinh viên có thể:

- Khởi tạo và cấu hình Microservice độc lập với Spring Boot 4.x kết nối MS SQL Server.
- Đăng ký và khám phá service qua Eureka Discovery Server.
- Thực hiện inter-service communication bằng Spring Cloud OpenFeign.
- Thiết kế đúng cấu trúc entity, schema SQL Server và REST API với input/output rõ ràng.
- Cấu hình Spring Cloud Gateway làm entry point, định tuyến và ghi log request.

Kiến trúc tổng quan



Thứ tự khởi chạy: Eureka (9001) → Student Service (8081) → Course Service (8082) → API Gateway (8080)

2. Bảng điểm tổng hợp

40đ Student Service	40đ Course Service	20đ API Gateway
-------------------------------	------------------------------	---------------------------

3. Phần 1 — Student Service (40 điểm)

Student Service

Port 8081 | DB: lab01_students | spring.application.name=student-service

Tên dự án: Tên Lớp_MSSV_StudentService

3.1 Khởi tạo project & cấu hình SQL Server

Mục	Yêu cầu	Ghi chú / Hint
1.1	Khởi tạo Spring Boot 4.x project với dependencies: spring-boot-starter-web, spring-boot-starter-data-jpa, mssql-jdbc, spring-cloud-starter-netflix-eureka-client, spring-boot-starter-validation.	<i>start.spring.io → chọn SQL Server Driver</i>
1.2	Cấu hình application.yml kết nối MS SQL Server: <pre>server.port: 8081 spring.application.name: student-service spring.datasource.url: jdbc:sqlserver://localhost:1433;databaseName=lab01_students spring.datasource.driver-class-name: com.microsoft.sqlserver.jdbc.SQLServerDriver spring.jpa.hibernate.ddl-auto: update spring.jpa.database-platform: org.hibernate.dialect.SQLServerDialect</pre>	<i>Tạo DB lab01_students trước</i>
1.3	Thêm @EnableDiscoveryClient vào main class. Kiểm tra student-service đăng ký thành công trên Eureka dashboard http://localhost:8761 .	<i>Status phải là UP</i>

3.2 Entity & Database Schema

Tạo entity Student ánh xạ vào bảng students trong SQL Server theo cấu trúc sau:

Field (Java)	Column (SQL)	Data Type	Constraints	Mô tả
id	id	BIGINT	PK, IDENTITY(1,1)	Khóa chính tự tăng
studentCode	student_code	NVARCHAR(20)	NOT NULL, UNIQUE	Mã sinh viên (vd: SV001)
fullName	full_name	NVARCHAR(150)	NOT NULL	Họ và tên đầy đủ
email	email	NVARCHAR(200)	NOT NULL, UNIQUE	Email sinh viên
phone	phone	NVARCHAR(20)	NULL	Số điện thoại
major	major	NVARCHAR(100)	NOT NULL	Ngành học
gpa	gpa	DECIMAL(3,2)	NULL, CHECK(0–4)	Điểm trung bình tích lũy
status	status	NVARCHAR(20)	DEFAULT 'ACTIVE'	ACTIVE / INACTIVE / GRADUATED
createdAt	created_at	DATETIME2	DEFAULT GETDATE()	Thời điểm tạo bản ghi
updatedAt	updated_at	DATETIME2	NULL	Thời điểm cập nhật gần nhất

Lưu ý: studentCode và email phải có ràng buộc UNIQUE trên database. Sử dụng @Column(unique=true) kết hợp @NotBlank trong Spring Validation. Field status mặc định là ACTIVE.

Mục	Yêu cầu	Ghi chú / Hint
1.4	Tạo entity Student với đầy đủ annotation JPA (@Entity, @Table, @Column) theo schema ở trên. Áp dụng Bean Validation (@NotBlank, @Email, @DecimalMin, @DecimalMax). Tạo StudentRepository extends JpaRepository<Student, Long> và bổ sung method findByStudentCode.	@Table(name="students") @Column(name="student_code", unique=true)

3.3 REST API — Input / Output chi tiết

Implement StudentController với đầy đủ 6 operations. Cấu trúc request/response theo bảng dưới:

Method	Endpoint	Mô tả	Request Body / Params	Response Body	HTTP
GET	/api/students <i>Hỗ trợ filter theo major, status</i>	Lấy tất cả sinh viên	Params (optional): major=<string> status=ACTIVE INACTIVE GRADUATED page=0, size=20	[{ id: Long, studentCode: String, fullName: String, email: String, major: String, gpa: Double, status: String }, ...]	200
GET	/api/students/{id}	Lấy sinh viên theo ID	PathVar: id (Long)	{ id, studentCode, fullName, email, phone, major, gpa, status, createdAt }	200 / 404
GET	/api/students/code/{code} <i>Dùng cho Feign từ Course Service</i>	Lấy theo studentCode	PathVar: code (String)	{ id, studentCode, fullName, email, major, status }	200 / 404
POST	/api/students <i>Validate unique studentCode, email</i>	Tạo sinh viên mới	{ studentCode: String (req), fullName: String (req), email: String (req), phone: String, major: String (req) }	{ id, studentCode, fullName, email, major, status, }	201 / 400 / 409

Method	Endpoint	Mô tả	Request Body / Params	Response Body	HTTP
				<pre> createdAt } </pre>	
PUT	<code>/api/students/{id}</code> <i>Không cho sửa studentCode</i>	Cập nhật thông tin	<pre> { fullName: String, email: String, phone: String, major: String, gpa: Double, status: String } </pre>	<pre> { id, studentCode, fullName, email, major, gpa, status, updatedAt } </pre>	200 / 404
DELETE	<code>/api/students/{id}</code> <i>Soft-delete: set status=INACTIVE</i>	Xóa sinh viên	PathVar: id (Long)	<pre> { message: "Student deactivated" } </pre>	200 / 404

Mục	Yêu cầu	Ghi chú / Hint
1.5	Implement StudentController với @RestController @RequestMapping("/api/students"). Trả đúng HTTP status code cho từng trường hợp. Xử lý lỗi duplicate studentCode/email bằng @ControllerAdvice trả về JSON { error, message, timestamp }.	ResponseEntity<?>, @Valid @RequestBody, GlobalExceptionHandler

4. Phần 2 — Course Service (40 điểm)

Course Service

Port 8082 | DB: lab01_courses | Giao tiếp sang Student Service qua OpenFeign

Tên dự án: TênLớp_MSSV_CourseService

4.1 Khởi tạo project & cấu hình SQL Server

Mục	Yêu cầu	Ghi chú / Hint
2.1	Khởi tạo project tương tự Phần 1, bổ sung spring-cloud-starter-openfeign. Cấu hình kết nối database lab01_courses trên port 8082. Thêm @EnableFeignClients vào main class.	Tạo database lab01_courses riêng biệt

4.2 Entities & Database Schema

Bảng courses

Field (Java)	Column (SQL)	Data Type	Constraints	Mô tả
id	id	BIGINT	PK, IDENTITY(1,1)	Khóa chính tự tăng

Field (Java)	Column (SQL)	Data Type	Constraints	Mô tả
courseCode	course_code	NVARCHAR(20)	NOT NULL, UNIQUE	Mã môn học (vd: CS101)
courseName	course_name	NVARCHAR(200)	NOT NULL	Tên môn học
credits	credits	INT	NOT NULL, CHECK(1–10)	Số tín chỉ
department	department	NVARCHAR(100)	NOT NULL	Khoa/Bộ môn phụ trách
description	description	NVARCHAR(MAX)	NULL	Mô tả nội dung môn học
maxStudents	max_students	INT	DEFAULT 50	Số sinh viên tối đa
createdAt	created_at	DATETIME2	DEFAULT GETDATE()	Thời điểm tạo bản ghi

Bảng enrollments

Field (Java)	Column (SQL)	Data Type	Constraints	Mô tả
id	id	BIGINT	PK, IDENTITY(1,1)	Khóa chính tự tăng
studentId	student_id	BIGINT	NOT NULL, FK*	ID sinh viên (ref sang student-service)
courseId	course_id	BIGINT	NOT NULL, FK(courses.id)	Khóa ngoại → courses
enrolledAt	enrolled_at	DATETIME2	DEFAULT GETDATE()	Ngày đăng ký
grade	grade	DECIMAL(4,1)	NULL, CHECK(0–10)	Điểm số (thang 10)
status	status	NVARCHAR(20)	DEFAULT 'ENROLLED'	ENROLLED / COMPLETED / DROPPED
note	note	NVARCHAR(500)	NULL	Ghi chú thêm

Lưu ý: studentId trong bảng enrollments KHÔNG phải là khóa ngoại thực sự trên SQL Server (vì dữ liệu sinh viên nằm ở service khác). Ràng buộc tính toàn vẹn được thực thi ở tầng ứng dụng qua Feign call.

Mục	Yêu cầu	Ghi chú / Hint
2.2	Tạo 2 entity Course và Enrollment với đầy đủ annotation JPA theo schema trên. Tạo CourseRepository và EnrollmentRepository với các custom query cần thiết: - findByStudentId(Long studentId) - findByCourseId(Long courseId) - countByCourseIdAndStatus(Long courseId, String status)	@Query hoặc Spring Data method naming

4.3 OpenFeign — Giao tiếp liên Service

Mục	Yêu cầu	Ghi chú / Hint
2.3	Tạo StudentClient với @FeignClient(name="student-service"). Khi tạo Enrollment mới: (1) Gọi GET	@FeignClient(name="student-service", fallback=...)

Mục	Yêu cầu	Ghi chú / Hint
	/api/students/{id} để xác minh sinh viên tồn tại và đang ACTIVE. (2) Nếu Feign throw exception → trả về HTTP 404 với message rõ ràng. (3) Kiểm tra số lượng sinh viên đã đăng ký không vượt maxStudents. <pre>@FeignClient(name = "student-service") public interface StudentClient { @GetMapping("/api/students/{id}") StudentDto getStudentById(@PathVariable Long id); }</pre>	

4.4 REST API — Input / Output chi tiết

Course Operations

Method	Endpoint	Mô tả	Request Body / Params	Response Body	HTTP
GET	/api/courses <i>Filter theo department</i>	Lấy danh sách môn học	Params (optional): department=<string> page=0, size=20	[{ id, courseCode, courseName, credits, department, maxStudents }, ...]	200
GET	/api/courses/{id}	Lấy môn học theo ID	PathVar: id (Long)	{ id, courseCode, courseName, credits, department, description, maxStudents, createdAt }	200 / 404
POST	/api/courses <i>Validate unique courseCode</i>	Tạo môn học mới	{ courseCode: String (req), courseName: String (req), credits: Int (req), department: String (req), description: String, maxStudents: Int }	{ id, courseCode, courseName, credits, department, createdAt }	201 / 400 / 409

Enrollment Operations

Method	Endpoint	Mô tả	Request Body / Params	Response Body	HTTP
POST	/api/enrollments <i>Validate studentId qua Feign; kiểm tra maxStudents</i>	Đăng ký học phần	{ studentId: }	{ id, studentId, }	201 / 400 /

Method	Endpoint	Mô tả	Request Body / Params	Response Body	HTTP
			Long (req), courseId: Long (req), note: String }	courseId, enrolledAt, status: 'ENROLLED', studentName: String, courseName: String }	404 / 409
GET	/api/enrollments/student/{studentId} <i>Trả về kèm thông tin môn học</i>	DS môn học của SV	PathVar: studentId (Long)	[{ id, courseId, courseName, credits, enrolledAt, grade, status }, ...]	200
GET	/api/enrollments/course/{courseId}	DS SV trong môn học	PathVar: courseId (Long)	[{ id, studentId, enrolledAt, grade, status }, ...]	200
PUT	/api/enrollments/{id}/grade <i>Chỉ cập nhật grade</i>	Cập nhật điểm	{ grade: Double (req) // 0.0 - 10.0 }	{ id, studentId, courseId, grade, status: 'COMPLETED' }	200 / 400 / 404
DELETE	/api/enrollments/{id} <i>Chỉ cho phép khi status=ENROLLED</i>	Hủy đăng ký	PathVar: id (Long)	{ message: "Enrollment dropped" }	200 / 400 / 404

Mục	Yêu cầu	Ghi chú / Hint
2.4	Implement CourseController và EnrollmentController theo đúng cấu trúc API ở trên. Response của POST /api/enrollments phải bao gồm studentName (lấy từ Feign) và courseName để client không cần gọi thêm. Xử lý lỗi 409 khi sinh viên đã đăng ký môn này rồi.	EnrollmentResponse DTO chứa cả studentName, courseName

5. Phần 3 — API Gateway (20 điểm)

API Gateway

Port 8080 | Spring Cloud Gateway (Reactive) | Load Balancer

Tên dự án: Tên Lớp_MSSSV_Gateway

Mục	Yêu cầu	Ghi chú / Hint
3.1	Khởi tạo project với dependencies: spring-cloud-starter-gateway, spring-cloud-starter-netflix-eureka-client, spring-boot-starter-actuator. KHÔNG thêm spring-boot-starter-web (conflict). Cấu hình port 8080, tên api-gateway, bật discovery locator.	<code>spring.cloud.gateway.discovery.locator.enabled=true</code>
3.2	Cấu hình route thủ công trong application.yml: <pre> spring.cloud.gateway.routes: - id: student-route uri: lb://student-service predicates: Path=/api/students/** - id: course-route uri: lb://course-service predicates: Path=/api/courses/** - id: enrollment-route uri: lb://course-service predicates: Path=/api/enrollments/** </pre>	<code>predicates: Path=/api/students/**</code>

6. Project structure requirements

#	Description		
1	The project must be Spring Boot with “jar” package type run on JDK 21 in IntelliJ format and the name must following convention in the project description		
2	User MS SQL Server Driver enclosed with Spring Boot Project in IntelliJ for MS SQL Server 2016 or later for the project		
3	The application must follow the MVC & Layer structural pattern. The package of java code must construct as the following		
		fu.<studentId>.<servicename>.entity	Contain entity classes
		fu.<studentId>.<servicename>.repository	Contain Repository Interface classes
		fu.<studentId>.<servicename>.service	Contain service Interface classes
		fu.<studentId>.<servicename>.service.impl	Contain service implementation classes
		fu.<studentId>.<servicename>.controller	Contain controller classes
		fu.<studentId>.<servicename>.dto	Contain data transfer objects
		fu.<studentId>.<servicename>.config	Contain configuration classes
		fu.<studentId>.<servicename>.common	Contain common classes

Note : Please replace <studentId> by your id in **lower case**, <servicename> by student, course, or gateway

7. Hướng dẫn kỹ thuật

Cấu hình MS SQL Server Driver (pom.xml)

```
<dependency>
  <groupId>com.microsoft.sqlserver</groupId>
  <artifactId>mssql-jdbc</artifactId>
  <version>12.4.2.jre11</version>
</dependency>
```

Spring Cloud BOM (pom.xml)

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>2025.0.0</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

Chuẩn bị SQL Server

Trước khi chạy service, tạo 2 database trên SQL Server:

```
CREATE DATABASE lab01_students;
CREATE DATABASE lab01_courses;
```

SQL Server phải chạy trên port 1433 (mặc định). Đảm bảo SQL Server Authentication được bật.

application.yml mẫu cho Microservice

```
spring:
  datasource:
    url:
jdbc:sqlserver://localhost:1433;databaseName=lab01_students;encrypt=true;trustServerCertificate=true
    username: sa
    password: <your_password>
    driver-class-name: com.microsoft.sqlserver.jdbc.SQLServerDriver
  jpa:
    hibernate.ddl-auto: none
    show-sql: true
    database-platform: org.hibernate.dialect.SQLServerDialect
```

```
eureka:  
  client:  
    service-url:  
      defaultZone: http://localhost:8761/eureka/
```

8. Hướng dẫn nộp bài

1. Build tất cả service thành công: `mvn clean package -DskipTests`.
2. Kiểm tra tất cả endpoint qua Gateway (port 8080) với Postman/curl.
3. Đảm bảo Eureka dashboard hiển thị đủ 3 service đang UP.
4. Nén source code: **Tên Lớp_MSSV_Lab01.zip** (Không bao gồm cả eureka-server nếu tự tạo).
5. Kèm file README.md: môi trường, thứ tự khởi chạy, SQL Server setup, API test examples.
6. Nộp lên Edunext trước khi hết giờ.

Chúc các bạn làm bài tốt!